

Aquarium Animal Census API — usage

Kanagavel A K · RAP pre-hackathon screening · repo: github.com/KanagavelAK/aquarium-rtdetr · weights: github.com/KanagavelAK/aquarium-rtdetr/releases/download/v1.0/best.pt

Quickstart

Python 3.10 to 3.13. No GPU needed to serve.

```
git clone https://github.com/KanagavelAK/aquarium-rtdetr
cd aquarium-rtdetr
pip install -r requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

On first start the API downloads `best.pt` from the GitHub release into `artifacts/`. Swagger UI is at <http://localhost:8000/docs>.

Tests (no GPU or checkpoint needed): `python -m pytest tests -q` — 20 tests covering routing, duplicate suppression, crowding, conflicts and the guardrail.

Local demo page (Gradio + API in one process): `python space_app.py` → <http://localhost:7860>.

Configuration

All settings are environment variables; `.env.example` lists them with defaults, and `uvicorn` picks up a `.env` automatically.

Variable	Default	Meaning
MODEL_WEIGHTS	<code>artifacts/best.pt</code>	checkpoint path
WEIGHTS_URL	GitHub release v1.0	where to fetch the checkpoint if the file is missing
WEIGHTS_KAGGLE_DATASET	empty	optional alternative source, owner/slug of a Kaggle dataset
CONF_THRESHOLD	<code>0.25</code>	default detection confidence cut-off (per-request override: form field confidence)
IMGSZ	<code>640</code>	inference size; must match training
DETECTOR_DEVICE	empty	empty lets Ultralytics choose; <code>cpu</code> pins inference to CPU
LOW_CONFIDENCE_FLOOR	<code>0.45</code>	guardrail: below this no assertion is made
CROWDING_MAX	<code>0.34</code>	guardrail: a count is refused above this crowding fraction
CONFIDENT_FRACTION_MIN	<code>0.5</code>	guardrail: a count needs this fraction of boxes above the floor
RANK_MARGIN_FRACTION	<code>0.15</code>	guardrail: a ranking needs the top two counts this far apart
ANTHROPIC_API_KEY	unset	optional; enables the language-model phrasing step
ANTHROPIC_MODEL	<code>claude-opus-5</code>	model for that single call
MAX_UPLOAD_BYTES	<code>12582912</code>	12 MB upload limit; larger files get a 400
LOG_LEVEL	<code>INFO</code>	one line per request with method, path, status, latency, request id

API

GET /health

```
{"status": "ok", "model_loaded": true, "weights": "artifacts/best.pt",
"classes": ["fish", "jellyfish", "penguin", "puffin", "shark", "starfish", "stingray"],
"llm_enabled": false}
```

POST /detect — image → boxes

```
curl -s -X POST http://localhost:8000/detect \
-F "file=@samples/tank_01.jpg" -F "confidence=0.25"
```

```
{
  "request_id": "25d6c213a3cd",
  "image": {
    "width": 1024,
    "height": 768
  },
  "confidence_threshold": 0.25,
```

```

"count": 51,
"counts_by_class": {
  "jellyfish": 49,
  "fish": 2
},
"detections": [
  {
    "label": "jellyfish",
    "confidence": 0.8886,
    "box_xyxy": [
      831.4,
      463.0,
      922.1,
      567.8
    ]
  },
  {
    "label": "jellyfish",
    "confidence": 0.8839,
    "box_xyxy": [
      747.4,
      162.6,
      852.7,
      251.1
    ]
  },
  {
    "label": "jellyfish",
    "confidence": 0.8774,
    "box_xyxy": [
      307.8,
      348.7,
      396.4,
      420.7
    ]
  },
  {
    "...": "48 more"
  }
],
"inference_ms": 3908.0
}

```

POST /ask — image + question → answer or refusal

```

curl -s -X POST http://localhost:8000/ask \
  -F "file=@samples/tank_01.jpg" -F "question=What is the most common animal here?"

```

```

{
  "request_id": "b65410fd64e4",
  "question": "What is the most common animal here?",
  "answer": "The most common animal is the jellyfish: 48 of the 50 animals detected, ahead of 2 fish.",
  "sufficient_information": true,
  "routing": {
    "needs_detection": true,
    "question_kind": "ranking",
    "targets": [],
    "rationale": "the question asks which animal is the most common, which needs reliable counts for every class present",
    "decided_by": "rules"
  },
  "detector_called": true,
  "guardrail_reasons": [],
  "answer_source": "template",
  "inference_ms": 696.6,
  "evidence": {
    "animals_detected": 50,
    "kinds_detected": 2,
    "counts": {

```

```

    "jellyfish": 48,
    "fish": 2
  },
  "ranking": [
    {
      "label": "jellyfish",
      "count": 48
    },
    {
      "label": "fish",
      "count": 2
    }
  ],
  "duplicates_suppressed": 1,
  "identity_conflicts": [],
  "crowding": 0.12,
  "classes": {
    "jellyfish": {
      "count": 48,
      "confident": 42,
      "tentative": 6,
      "max_confidence": 0.8886,
      "crowding": 0.125,
      "conflicts": 0
    },
    "fish": {
      "count": 2,
      "confident": 2,
      "tentative": 0,
      "max_confidence": 0.6147,
      "crowding": 0.0,
      "conflicts": 0
    }
  },
  "...": "animals and raw detections omitted here"
}

```

The honest refusal — samples/tank_02.jpg, a reef tank full of small fish:

```

curl -s -X POST http://localhost:8000/ask \
-F "file=@samples/tank_02.jpg" -F "question=How many fish are in this image?"

```

```

{
  "question": "How many fish are in this image?",
  "answer": "I do not have enough information to answer that confidently. Only 23 of the 67 fish detections scored at or above 0.45, so most of that count rests on weak evidence.",
  "sufficient_information": false,
  "routing": {
    "needs_detection": true,
    "question_kind": "count",
    "targets": [
      "fish"
    ]
  },
  "rationale": "the question asks for a count of fish in the image",
  "decided_by": "rules"
},
"detector_called": true,
"guardrail_reasons": [
  "only 23 of the 67 fish detections scored at or above 0.45, so most of that count rests on weak evidence"
],
"answer_source": "template",
"evidence": {
  "counts": {
    "fish": 67,
    "shark": 1
  }
},
"duplicates_suppressed": 29,

```

```

    "crowding": 0.206,
    "classes": {
      "fish": {
        "count": 67,
        "confident": 23,
        "tentative": 44,
        "max_confidence": 0.7946,
        "crowding": 0.209
      }
    },
    "...": "..."
  }
}

```

Questions that never reach the detector return `detector_called: false` with the routing rationale, e.g. *What species of fish is that?* → `question_kind: out_of_detector_scope`; *What is the capital of France?* → `question_kind: not_about_the_image`. A question that needs an image but arrives without one gets a 400 with a plain message.

Deployment

Docker (CPU):

```

docker build -t aquarium-rt detr .
docker run -p 8000:8000 aquarium-rt detr

```

The image installs CPU-only torch, fetches `best.pt` on first start and has a `/health` healthcheck.

Demo page. `python space_app.py` serves a Gradio page at `http://localhost:7860` with the FastAPI routes mounted in the same process. The file is also a ready Hugging Face Space entry point (Gradio SDK; on Spaces the API is mounted under `/api/` and inference is pinned to CPU), so `README.md`'s front matter plus `space_app.py` deploy as-is when a Space is available.

Logging and errors. Every request gets an `x-request-id` header and one log line with method, path, status and latency. Bad uploads, oversized files and questions that need an image but have none return a 400 with a plain message; unexpected inference errors return a 500 with the request id and a full trace in the log.